

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 1
по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ ресурсной эффективности алгоритмов

Студент Крылов В.С.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л.

Преподаватели Строганов Д.В.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**
1.1 Постановка задачи 5
1.2 Вывод 5

2 Конструкторская часть **6**
2.1 Требования к программному обеспечению 6
2.2 Разработка алгоритмов 6
2.2.1 Поиск полным перебором 6
2.2.2 Нерекурсивный бинарный поиск 7
2.2.3 Рекурсивный бинарный поиск 11
2.3 Вывод 12

3 Технологическая часть **13**
3.1 Средства разработки 13
3.2 Реализация алгоритмов 13
3.3 Тестирование реализаций 15
3.4 Вывод 16

4 Исследовательская часть **17**
4.1 Оценка временной эффективности 17
4.1.1 Поиск полным перебором 17
4.1.2 Бинарный поиск 17
4.2 Оценка затрачиваемой памяти 19
4.2.1 Линейный поиск 19
4.2.2 Нерекурсивный бинарный поиск 19
4.2.3 Рекурсивный бинарный поиск 19
4.3 Эмпирическая проверка временной эффективности 20
4.4 Сравнительный анализ 21
4.5 Вывод 22

ЗАКЛЮЧЕНИЕ **23**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **24**

ПРИЛОЖЕНИЕ А **25**

ВВЕДЕНИЕ

Целью работы является выполнение сравнительного анализа ресурсной эффективности алгоритмов поиска элементов в массиве.

Для достижения поставленной цели были определены следующие задачи:

— описать решаемую согласно варианту задачу;

— разработать рекурсивный и нерекурсивные алгоритмы решения задачи, согласно варианту;

— описать разработанные алгоритмы схемами алгоритмов;

— описать требования к программному обеспечению;

— описать средства реализации алгоритмов;

— реализовать разработанные алгоритмы;

— выполнить тестирование реализации алгоритмов;

— выполнить теоретическую оценку затрачиваемой реализацией каждого алгоритма памяти (для рекурсивного алгоритма — на материале анализа высоты дерева рекурсивных вызовов и оценки затрачиваемой на один вызов функции памяти);

— оценить временную эффективность алгоритмов или их реализаций в лучшем и худшем случае в терминах числа сравнений искомой величины с другими величинами;

— представить результаты сравнительного анализа в формате таблицы;

— сделать выводы из сравнительного анализа реализации разработанных алгоритмов решения одной и той же задачи, заданной вариантом, по критериям временной и ёмкостной эффективности, а также дополнительных требований, налагаемых алгоритмом.

1 Аналитическая часть

1.1 Постановка задачи

В данной лабораторной работе решается задача поиска элемента в массиве. Дан массив уникальных целых чисел. Для заданного значения требуется определить, содержится ли искомое значение в массиве, и если содержится — вернуть его индекс в массиве.

Для решения задачи требуется разработать следующие алгоритмы:

— поиск полным перебором;
— нерекурсивный бинарный поиск, вариант реализации 1;
— нерекурсивный бинарный поиск, вариант реализации 2;
— рекурсивный бинарный поиск.

1.2 Вывод

В результате аналитической части была поставлена задача поиска элемента в массиве и определены алгоритмы для её решения.

2 Конструкторская часть

2.1 Требования к программному обеспечению

К разрабатываемому программному обеспечению предъявляются следующие требования:

а) ввод размера массива N (натуральное число);

б) автоматическое наполнение массива A N уникальными целочисленными значениями;

в) создание дубликата массива A;

г) сортировка элементов дубликата по возрастанию;

д) циклический вывод интерактивного меню с возможностью выбора и выполнения следующих операций:

1) вывод отсортированного дубликата массива A;
2) выбор используемого алгоритма поиска;
3) ввод искомого значения $x \in \mathbb{Z}$;
4) вывод индекса элемента, равного x, или -1, если элемент отсутствует;
5) выход из программы.

2.2 Разработка алгоритмов

Для каждого алгоритма определены:

— A — входной массив уникальных целых чисел;

— N — количество элементов массива A;

— x — искомый элемент;

— $index$ — индекс найденного элемента внутри массива A (значение -1 при отсутствии).

Для алгоритмов бинарного поиска дополнительно определено:

— l — индекс левой границы обрабатываемой части массива A;

— r — индекс правой границы обрабатываемой части массива A;

— m — индекс серединного элемента обрабатываемой части массива A.

2.2.1 Поиск полным перебором

При алгоритме поиска полным перебором в массиве A последовательно перебираются его элементы от индекса 0 до $N-1$ и сравниваются с искомым

значением. При совпадении поиск завершается с результатом — индексом найденного элемента. В случае перебора всех элементов массива и отсутствии совпадения результат приравнивается к -1. Схема алгоритма приведена на рисунке 2.1.

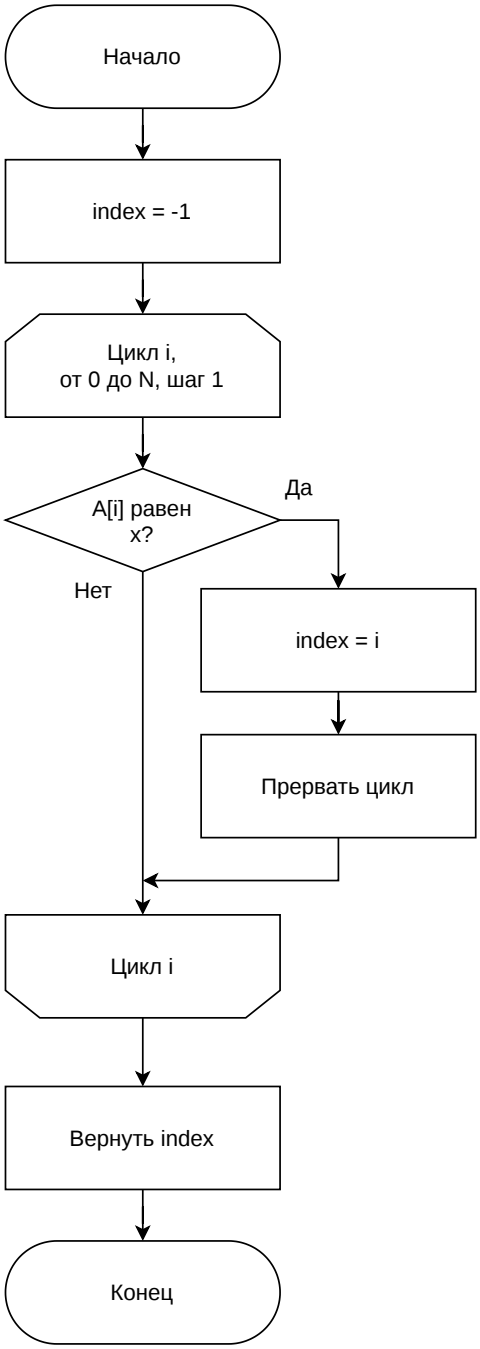


Рисунок 2.1 — Схема алгоритма поиска полным перебором

2.2.2 Нерекурсивный бинарный поиск

При нерекурсивной реализации алгоритма бинарного поиска рассматривается массив, отсортированный по возрастанию. Вводятся переменные, обо-

значающие границы обрабатываемой части массива. Изначально $l = 0$, $r = N - 1$. На каждой итерации цикла внутри поиска определяется средний элемент массива с индексом $m = l + (r - l)/2$. Для следующей итерации определяются новые границы обрабатываемой части массива с помощью сравнения среднего элемента с искомым значением.

Поиск продолжается до тех пор, пока $l < r$. После завершения цикла элемент $A[l]$ сравнивается с искомым: при совпадении результатом поиска является индекс l, иначе искомый элемент в массиве отсутствует.

Схемы нерекурсивных алгоритмов бинарного поиска приведены на рисунках 2.2 и 2.3.

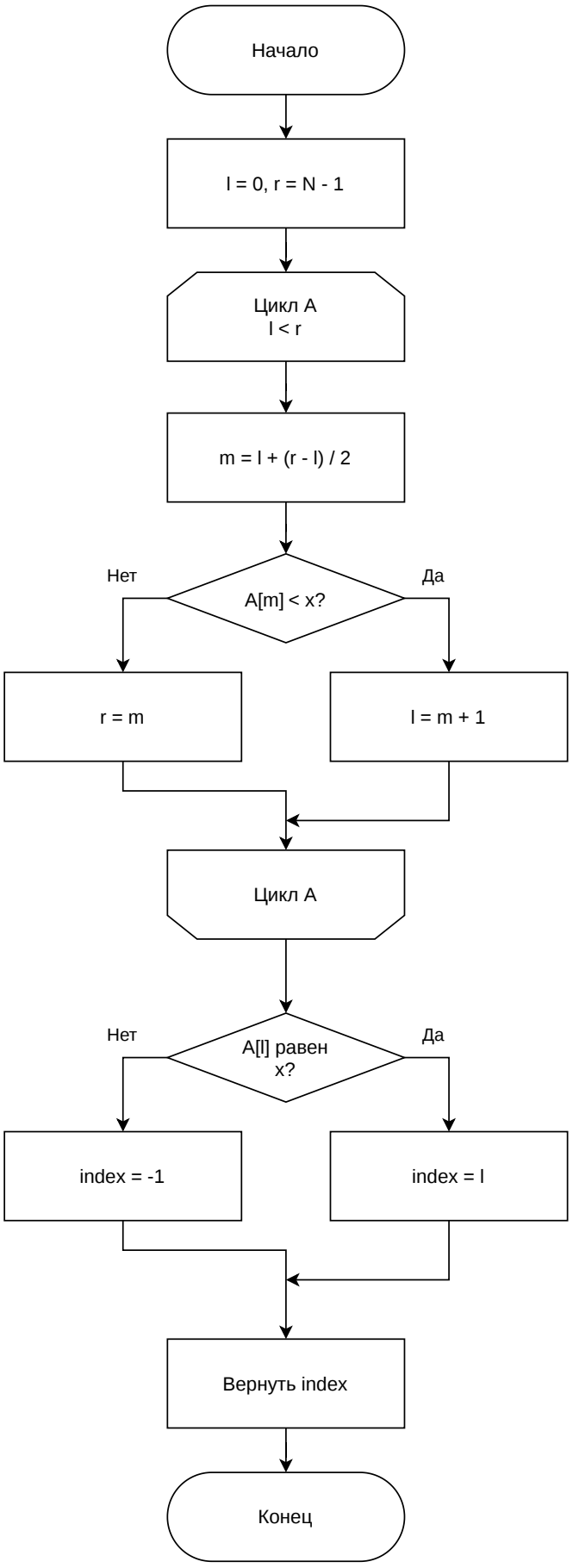


Рисунок 2.2 — Схема нерекурсивного алгоритма бинарного поиска с отложенным выходом

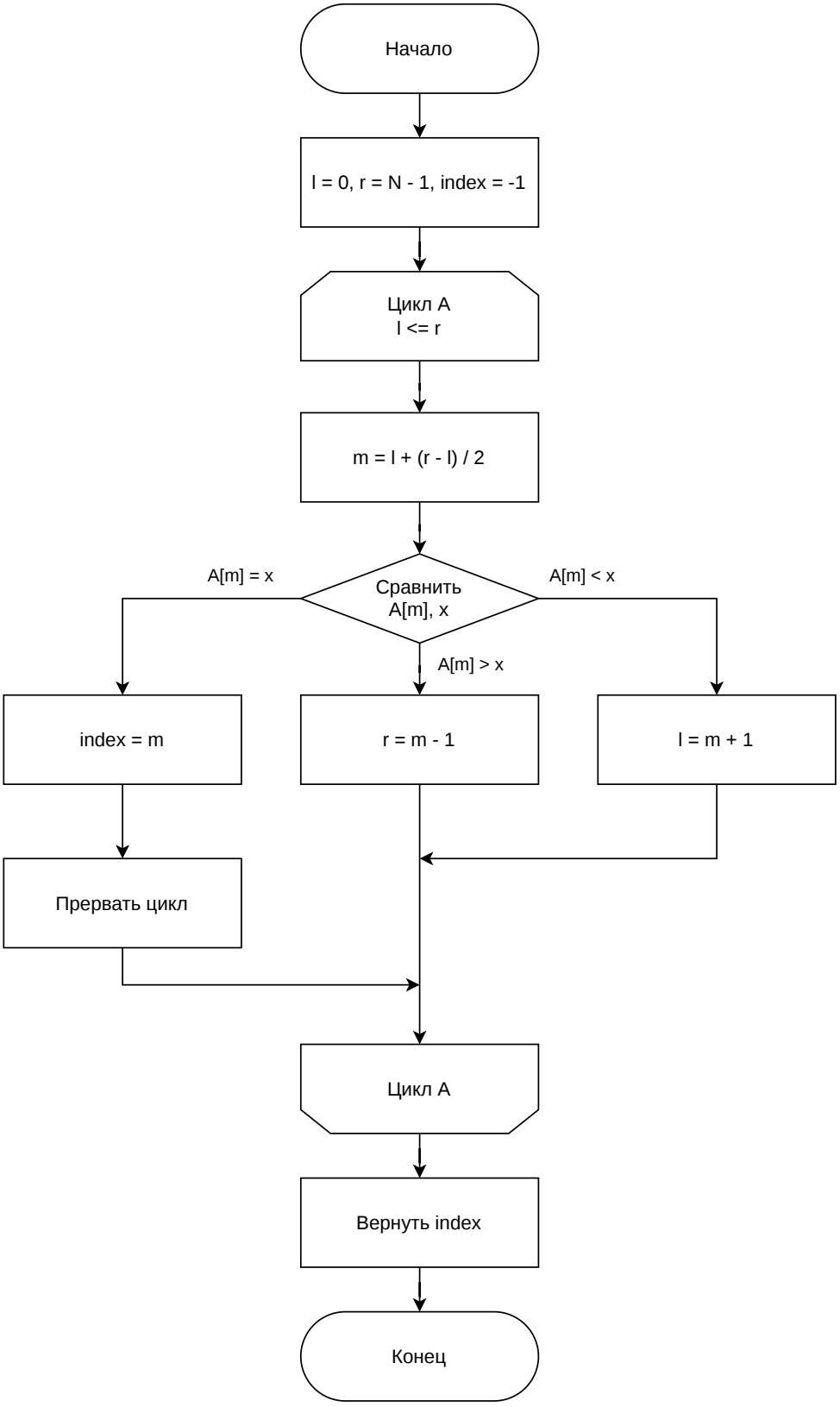


Рисунок 2.3 — Схема нерекурсивного алгоритма бинарного поиска с предварительным выходом

2.2.3 Рекурсивный бинарный поиск

При рекурсивной реализации алгоритма бинарного поиска реализуется аналогичный принцип деления обрабатываемой части массива пополам, что и нерекурсивные реализации, однако организуется рекурсивный вызов функции с новыми границами обрабатываемой части. На рисунке 2.4 представлена схема рекурсивной процедуры бинарного поиска.

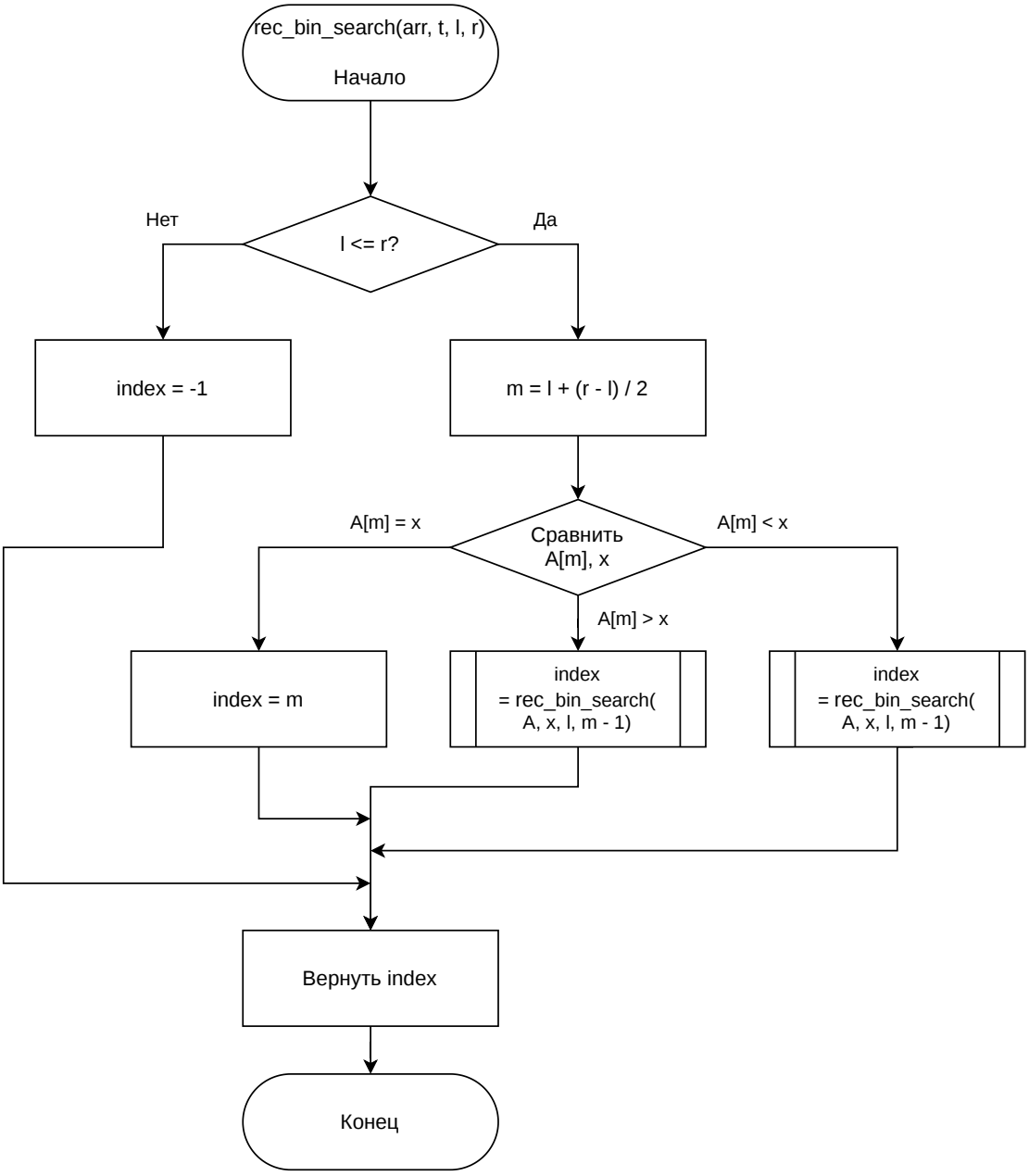


Рисунок 2.4 — Схема рекурсивного алгоритма бинарного поиска

2.3 Вывод

В результате конструкторской части были определены требования к программному обеспечению, были описаны с помощью схем алгоритмов варианты реализации алгоритма поиска элемента в массиве.

3 Технологическая часть

3.1 Средства разработки

В качестве языка программирования со статической типизацией был выбран язык C++ [1] (стандарт C++20 [2]). Для массива использовался контейнер стандартной библиотеки `std::vector<int>` [1].

3.2 Реализация алгоритмов

На рисунке 3.1 представлена программная реализация алгоритма поиска полным перебором.

```
int linear_search(const vector<int>& A, int x)
{
    int index = -1;
    for (size_t i = 0; i < A.size(); ++i)
    {
        if (A[i] == x)
        {
            index = i;
            break;
        }
    }

    return index;
}
```

Рисунок 3.1 — Поиск полным перебором

На рисунках 3.2 и 3.3 представлены два варианта программных реализаций нерекурсивного алгоритма бинарного поиска.

```
int bin_search_1(const vector<int>& A, int x)
{
    if (A.empty())
        return -1;

    int l = 0, r = A.size() - 1, m;

    while (l < r)
    {
        m = l + (r - l) / 2;
        if (A[m] < x)
            l = m + 1;
        else
            r = m;
    }

    if (l < A.size() && A[l] == x)
        return l;

    return -1;
}
```

Рисунок 3.2 — Нерекурсивный бинарный поиск с отложенным выходом

```
int bin_search_2(const vector<int>& A, int x)
{
    int l = 0, r = A.size() - 1, m;

    while (l <= r)
    {
        m = l + (r - l) / 2;

        if (A[m] == x)
            return m;
        if (A[m] < x)
            l = m + 1;
        else
            r = m - 1;
    }

    return -1;
}
```

Рисунок 3.3 — Нерекурсивный бинарный поиск с предварительным выходом

На рисунках 3.4 и 3.5 представлены программная реализация рекурсивного алгоритма бинарного поиска и соответствующая функция-обёртка.

```
int recursion_bin_search(const vector<int>& A, int x, int l, int r)
{
    if (l > r)
        return -1;

    int m = l + (r - l) / 2;

    if (A[m] == x)
        return m;

    if (A[m] < x)
        return recursion_bin_search(A, x, m + 1, r);
    return recursion_bin_search(A, x, l, m - 1);
}
```

Рисунок 3.4 — Рекурсивный бинарный поиск

```
int recursion_bin_search(const vector<int>& A, int x)
{
    if (A.empty())
        return -1;
    return recursion_bin_search(A, x, 0, A.size() - 1);
}
```

Рисунок 3.5 — Оберточная функция для рекурсивного бинарного поиска

3.3 Тестирование реализаций

В таблице 3.1 представлены результаты функционального тестирования разработанных реализаций алгоритмов поиска.

Таблица 3.1 — Функциональные тесты

№	Входные данные		Выходные данные		Комментарий
	Массив	x	Ожидаемые	Полученные	
1	[10]	10	0	0	Один элемент, совпадение
2	[10]	5	-1	-1	Один элемент, не найден
3	[1, 3, 5, 7, 9]	1	0	0	Первый элемент, нечётная длина
4	[1, 3, 5, 7, 9]	9	4	4	Последний элемент, нечётная длина
5	[1, 3, 5, 7, 9]	5	2	2	Середина, нечётная длина
6	[1, 3, 5, 7, 9]	0	-1	-1	Не найден, нечётная длина
7	[1, 3, 5, 7]	1	0	0	Первый элемент, чётная длина
8	[1, 3, 5, 7]	7	3	3	Последний элемент, чётная длина
9	[1, 3, 5, 7]	5	2	2	Середина, чётная длина
10	[1, 3, 5, 7]	0	-1	-1	Не найден, чётная длина

Все разработанные варианты реализации алгоритма поиска успешно прошли функциональное тестирование.

3.4 Вывод

В результате технологической части были выбраны язык программирования и средства разработки, представлены листинги исходного кода разработанных алгоритмов поиска, выполнено функциональное тестирование, результаты которого подтвердили корректность работы всех реализаций.

4 Исследовательская часть

4.1 Оценка временной эффективности

Мерой временной эффективности рассматриваемых алгоритмов выступает количество операций сравнения искомого значения x с элементами массива A.

4.1.1 Поиск полным перебором

Лучший случай реализуется, если искомый элемент расположен на первой позиции массива. В этом случае алгоритм выполняет единственное сравнение, что даёт:

$$T_{\min}(N) = 1 = O(1). \tag{4.1}$$

Худший случай реализуется, если искомое значение отсутствует в массиве либо находится на последней позиции. Тогда требуется последовательно проверить все N элементов:

$$T_{\max}(N) = N = O(N). \tag{4.2}$$

4.1.2 Бинарный поиск

Принцип бинарного поиска основан на последовательном уменьшении области поиска вдвое. На каждом шаге алгоритм сравнивает искомое значение со средним элементом текущего обрабатываемой части.

N_i — размер обрабатываемой части после i-й итерации, при этом $N_0 = N$. Рекуррентное соотношение имеет вид:

$$\begin{aligned} N_1 &= \frac{N}{2}, \\ N_2 &= \frac{N}{2^2}, \\ &\vdots \\ N_i &= \frac{N}{2^i}. \end{aligned} \tag{4.3}$$

Процесс завершается при достижении обрабатываемой части размера единица: $N/2^i = 1$, откуда $i = \log_2 N$. Количество итераций ограничено логариф-

мической функцией, а итоговое число сравнений определяется структурой конкретного варианта реализации.

Нерекурсивный вариант с отложенным выходом

В данной реализации цикл продолжается до тех пор, пока границы диапазона не совпадут ($l = r$). На каждой итерации выполняется единственное сравнение $A[m] < x$, определяющее направление уменьшения обрабатываемой части. После завершения цикла производится финальная проверка $A[l] == x$.

Поскольку досрочное завершение отсутствует, лучший и худший случаи совпадают. Общее число сравнений составляет:

$$T_{\min}(N) = T_{\max}(N) = \lfloor \log_2 N \rfloor + 1 = O(\log_2 N). \tag{4.4}$$

Нерекурсивный вариант с досрочным выходом

На каждой итерации данного алгоритма выполняются два сравнения: сначала проверяется равенство $A[m] == x$, затем — порядок $A[m] < x$. При обнаружении совпадения поиск немедленно завершается.

Лучший случай — искомый элемент совпадает со средним элементом исходного массива. В этом случае достаточно одного сравнения:

$$T_{\min}(N) = 1 = O(1). \tag{4.5}$$

Худший случай — элемент отсутствует или расположен на границе обрабатываемой части. Тогда на каждой из $\lceil \log_2 N \rceil + 1$ итераций выполняются оба сравнения:

$$T_{\max}(N) = 2 \cdot (\lfloor \log_2 N \rfloor + 1) = O(\log_2 N). \tag{4.6}$$

Рекурсивный вариант

Рекурсивная реализация эквивалентна нерекурсивному варианту с досрочным выходом по числу выполняемых сравнений. Каждый рекурсивный вызов содержит две операции сравнения ($A[m] == x$ и $A[m] < x$), а глубина рекурсии

ограничена величиной $\lceil \log_2 N \rceil + 1$:

$$T_{\min}(N) = 1 = O(1),$$
$$T_{\max}(N) = 2 \cdot (\lfloor \log_2 N \rfloor + 1) = O(\log_2 N). \tag{4.7}$$

4.2 Оценка затрачиваемой памяти

Оценка памяти производится только для дополнительных структур данных, не включая входной массив.

4.2.1 Линейный поиск

Алгоритм оперирует двумя служебными переменными:

— i — счётчик текущего элемента;

— $index$ — переменная для хранения результата.

Функция вызывается однократно, дополнительные затраты памяти не зависят от размера входных данных: $O(1)$.

4.2.2 Нерекурсивный бинарный поиск

Оба нерекурсивных варианта используют три вспомогательные переменные:

— l — левая граница обрабатываемой части;

— r — правая граница обрабатываемой части;

— m — индекс среднего элемента обрабатываемой части.

Так как количество переменных фиксировано и не растёт с увеличением N, дополнительная память составляет $O(1)$.

4.2.3 Рекурсивный бинарный поиск

Каждый рекурсивный вызов создаёт в стеке новый кадр, содержащий параметры функции (ссылку на массив A, значение x, границы l и r), локальную переменную m, адрес возврата и служебные данные. Объём одного кадра — M_s.

Максимальная глубина рекурсии определяется скоростью сужения диапазона и равна $\lfloor \log_2 N \rfloor + 1$. Суммарные затраты памяти в наихудшем случае:

$$M_{\text{rec}}(N) = (\lfloor \log_2 N \rfloor + 1) \cdot M_s = O(\log_2 N). \tag{4.8}$$

Следовательно, рекурсивная реализация требует логарифмического объёма дополнительной памяти, в то время как нерекурсивные варианты обходятся константным.

4.3 Эмпирическая проверка временной эффективности

Для подтверждения корректности полученных теоретических оценок была проведена серия эмпирических замеров количества операций сравнения. Модифицированные реализации алгоритмов, содержащие встроенные счётчики сравнений, приведены в приложении А.

Результаты замеров для различных размеров массива N в лучшем и худшем случаях сведены в таблицу 4.1.

Таблица 4.1 — Эмпирические замеры количества сравнений

Размер N	**Случай**	**Линейный**	**Бин. 1**	**Бин. 2**	**Бин. Рек.**
2	Лучший	1	2	1	1
	Худший	2	2	4	4
4	Лучший	1	3	1	1
	Худший	4	3	6	6
8	Лучший	1	4	1	1
	Худший	8	4	8	8
16	Лучший	1	5	1	1
	Худший	16	5	10	10
64	Лучший	1	7	1	1
	Худший	64	7	14	14
256	Лучший	1	9	1	1
	Худший	256	9	18	18
1024	Лучший	1	11	1	1
	Худший	1024	11	22	22
4096	Лучший	1	13	1	1
	Худший	4096	13	26	26

Анализ данных таблицы показывает полное совпадение эмпирических результатов с теоретическими формулами, выведенными ранее. В частности:

— для бинарного поиска с отложенным выходом (Бин. 1) количество сравнений в лучшем и худшем случаях совпадает и равно $1 + \log_2 N$;

— для варианта с досрочным выходом (Бин. 2) и рекурсивной реализации в худшем случае количество сравнений ровно в два раза превышает количество итераций, что полностью подтверждает формулу $2 \cdot (1 + \log_2 N)$.

4.4 Сравнительный анализ

Результаты сравнения алгоритмов представлены в таблице 4.2, где ЛС и ХС обозначают наилучший и наихудший случаи соответственно.

Таблица 4.2 — Сравнение алгоритмов поиска

Критерии / **Алгоритм**	**Оценка временной эффективности как функции от числа необходимых сравнений**		**Требования, налагаемые алгоритмом**	**Оценка памяти, затрачиваемой алгоритмом сверх входных данных**
	ЛС	**ХС**		
Поиск полным перебором	1	N	Нет	$i \to O(1)$, 1 вызов функции
Нерекурсивный бинарный поиск (отложенный выход)	$1 + \log_2 N$	$1 + \log_2 N$	Массив предварительно отсортирован и поддерживается отсортированным	$l, r, m \to O(1)$, 1 вызов функции
Нерекурсивный бинарный поиск (предв. выход)	1	$2 \cdot (1 + \log_2 N)$		
Рекурсивный бинарный поиск	1	$2 \cdot (1 + \log_2 N)$		$\log_2 N \cdot M_s \to O(\log_2 N)$, $\log_2 N$ вызовов

Выводы по таблице 4.2:

— Линейный поиск в наихудшем случае требует N сравнений, однако не предъявляет требований к упорядоченности данных.

— Бинарный поиск обеспечивает логарифмическую сложность $O(\log_2 N)$, но требует предварительную сортировку массива и поддержание его в отсортированном состоянии.

— Вариант реализации с досрочным выходом достигает константной сложности $O(1)$ в лучшем случае, однако в худшем случае выполняет вдвое больше сравнений по сравнению с вариантом с отложенным выходом.

— Рекурсивная реализация демонстрирует ту же временную эффективность, что и нерекурсивный вариант с досрочным выходом, но затрачивает $O(\log_2 N)$ дополнительной памяти на организацию стека вызовов.

Следует отметить дополнительное ограничение рекурсивной реализации: глубина рекурсии ограничена размером стека вызовов, предоставляемого операционной системой.

— Для поиска в неотсортированном массиве оптимален линейный поиск. Для отсортированных данных при $N \gg 1$ предпочтителен нерекурсивный бинарный поиск с отложенным выходом: он гарантирует минимальное число сравнений в наихудшем случае ($\approx \log_2 N$).

4.5 Вывод

В результате исследовательской части были проведены оценки временной эффективности и затрачиваемой памяти для каждого алгоритма поиска. Был проведён сравнительный анализ полученных результатов.

ЗАКЛЮЧЕНИЕ

В процессе выполнения лабораторной работы поставленная цель была достигнута: осуществлен теоретический и практический сравнительный анализ ресурсных затрат и временной сложности алгоритма полного перебора и трех различных реализаций бинарного поиска.

Все задачи были решены:

— описана решаемая согласно варианту задача;

— разработаны рекурсивный и нерекурсивные алгоритмы решения задачи, согласно варианту;

— описаны разработанные алгоритмы схемами алгоритмов;

— описаны требования к программному обеспечению;

— описаны средства реализации алгоритмов;

— реализованы разработанные алгоритмы;

— выполнено тестирование реализации алгоритмов;

— выполнена теоретическая оценка затрачиваемой реализацией каждого алгоритма памяти (для рекурсивного алгоритма — на материале анализа высоты дерева рекурсивных вызовов и оценки затрачиваемой на один вызов функции памяти);

— оценена временная эффективность алгоритмов или их реализаций в лучшем и худшем случае в терминах числа сравнений искомой величины с другими величинами;

— представлены результаты сравнительного анализа в формате таблицы;

— сделаны выводы из сравнительного анализа реализации разработанных алгоритмов решения одной и той же задачи, заданной вариантом, по критериям временной и ёмкостной эффективности, а также дополнительных требований, налагаемых алгоритмом.

В результате исследования установлено, что выбор метода поиска определяется соотношением между частотой поисковых запросов и затратами на поддержание упорядоченности данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Страуструп, Б. Программирование: принципы и практика с использованием C++ : пер. с англ. / Б. Страуструп. — 2-е изд. — М. : ООО «И.Д. Вильямс», 2016. — 1328 с.
2. ISO/IEC 14882:2020. Programming languages — C++. — International Organization for Standardization, 2020.

ПРИЛОЖЕНИЕ А

Листинг А — Листинг алгоритмов с замером количества сравнений

```
static int linear_search(const vector<int>& A, int x, size_t* count = nullptr)
{
    int index = -1;
    for (size_t i = 0; i < A.size(); ++i)
    {
        if (count)
            (*count)++;
        if (A[i] == x)
        {
            index = i;
            break;
        }
    }
    return index;
}

static int bin_search_1(const vector<int>& A, int x, size_t* count = nullptr)
{
    if (A.empty())
        return -1;

    int l = 0, r = static_cast<int>(A.size()) - 1, m;

    while (l < r)
    {
        m = l + (r - l) / 2;
        if (count)
            (*count)++;
        if (A[m] < x)
            l = m + 1;
        else
            r = m;
    }

    if (count)
        (*count)++;
    if (A[l] == x)
        return l;
```

```
    return -1;
}

static int bin_search_2(const vector<int>& A, int x, size_t* count = nullptr)
{
    if (A.empty())
        return -1;

    int l = 0, r = static_cast<int>(A.size()) - 1, m;

    while (l <= r)
    {
        m = l + (r - l) / 2;

        if (count)
            (*count)++;
        if (A[m] == x)
            return m;

        if (count)
            (*count)++;
        if (A[m] < x)
            l = m + 1;
        else
            r = m - 1;
    }

    return -1;
}

static int recursion_bin_search(const vector<int>& A, int x, int l, int r,
    size_t* count = nullptr)
{
    if (l > r)
        return -1;

    int m = l + (r - l) / 2;

    if (count)
        (*count)++;
    if (A[m] == x)
```

```
        return m;

    if (count)
        (*count)++;
    if (A[m] < x)
        return recursion_bin_search(A, x, m + 1, r, count);

    return recursion_bin_search(A, x, l, m - 1, count);
}

static int recursion_bin_search(const vector<int>& A, int x, size_t* count =
    nullptr)
{
    if (A.empty())
        return -1;
    return recursion_bin_search(A, x, 0, static_cast<int>(A.size()) - 1,
        count);
}
```